



ANSWERED



CORRECT



INCORRECT



FLAGGED

0 / 70

0%

SUBMIT ALL & SCORE

REVEAL ALL ANSWERS

RESET EXAM

PRINT / SAVE PDF

NEXT UNANSWERED ↓

ALL (70)

COMPOSITION (16)

CONTENT AGGREGATION (7)

CUSTOMIZING USD (4)

DATA EXCHANGE (11)

DATA MODELING (9)

DEBUGGING (8)

PIPELINE DEV (10)

VISUALIZATION (5)

COMPOSITION

#1

COMPOSITION · 23% EXAM WEIGHT

A USD file has two root prims: `/Props` and `/Lights`. No `defaultPrim` metadata is set. A downstream stage adds a reference to this file without specifying a prim path. What happens?

A

USD composes both `/Props` and `/Lights` under the referencing prim

B

USD picks the first root prim alphabetically as the default

C

The reference fails to resolve—USD cannot determine which prim to use without a defaultPrim or explicit prim path

D

USD merges all root prims into a single Scope prim under the referencing path

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When a layer has no **defaultPrim** metadata and a reference does not specify an explicit prim path, USD cannot determine which prim to use as the root of the reference. The



reference **fails silently or raises a composition error**—no content is composed. Always set defaultPrim on referenceable assets.

#2

COMPOSITION • 23% EXAM WEIGHT

A character asset uses `specializes` to inherit from a `\_class\_Character` class prim. A downstream shot layer adds a local opinion to the character's `primvars:displayColor`. According to LIVERPS, what is the result?

- ☐ A The class prim's value wins because specializes has higher strength than local opinions
- ☒ B The local opinion in the shot layer wins because local opinions (L) are the strongest arc in LIVERPS
- ☐ C The specializes arc and local opinion are merged into an averaged value
- ☐ D USD raises an error when a local opinion conflicts with a specializes arc

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In LIVERPS, **Local opinions (L)** are the strongest. Even though `specializes` is used for 'persistent refinements,' a *stronger layer's local opinion* still takes precedence over what flows in from the specializes arc. Local opinions always win in LIVERPS regardless of which composition arc is on the other side.

#3

COMPOSITION • 23% EXAM WEIGHT

A prop asset has the following reference list: `prepend references = [ @./geo.usda@, @./materials.usda@ ]`. Both layers define the attribute `double3 xformOp:translate` on the same prim. Which value wins?

- ☐ A The value from `materials.usda` because it appears later in the list
- ☒ B The value from `geo.usda` because it appears first (prepended) in the list and therefore has stronger opinion
- ☐ C Both values are averaged to produce the final composed value

D

USD raises a conflict error and uses a built-in default value

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Within a reference list, layers listed **earlier have stronger opinions**. ``geo.usda`` appears first in the prepended reference list, so its ``xformOp:translate`` opinion is stronger than ``materials.usda``'s. This is why ordering in composition arc lists matters and should be intentional.

#4

COMPOSITION · 23% EXAM WEIGHT



An artist wants to apply a global color grading override to every renderable prim in a stage without modifying any individual asset files. Which composition mechanism allows opinions to be broadcast across the entire scene graph?

A

A variant set defined on the stage's default prim

B

A class prim (defined with ``class`` specifier) used with ``inherits`` arcs on all relevant prims

C

A payload arc in the root layer pointing to the override file

D

A ``specializes`` arc from the root prim of the stage

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Class prims with inherits** are the canonical mechanism for broadcasting opinions to many prims. By defining a ``/_class_RenderableOverride`` class prim and having all renderable prims inherit from it, any change to the class prim is automatically reflected in all inheriting prims. This is how 'global look' overrides are typically implemented.

#5

COMPOSITION · 23% EXAM WEIGHT



Which of the following is the correct ordering of composition arcs from STRONGEST to WEAKEST?

A

Local → Inherits → Variants → References → Payloads → Specializes

B Specializes → Payloads → References → Variants → Inherits → Local

C Inherits → Local → Variants → Specializes → References → Payloads

D Local → Variants → Inherits → Payloads → References → Specializes

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: LIVERPS: **L**ocal → **I**nherits → **V**ariants → **E**xternally referenced → **P**ayloads → **S**pecializes. Local is always strongest; Specializes is always weakest. Payloads and References are both external composition arcs, with References (E) stronger than Payloads (P).

#6

COMPOSITION • 23% EXAM WEIGHT

A stage has the following sublayer order (top to bottom): `shot\_anim.usda`, `shot\_layout.usda`, `asset\_lib.usda`. The same attribute is authored in all three. Which layer's value appears on the composed stage?

A `asset\_lib.usda` — the base layer always wins as the source of truth

B `shot\_layout.usda` — it was authored after the asset layer

C `shot\_anim.usda` — it is the topmost layer and therefore has the strongest opinion

D All three values are composed using an average weighted by layer position

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Sublayers are ordered from strongest to weakest as listed. The **topmost layer has the strongest opinion**. `shot\_anim.usda` is listed first (top), so its attribute value wins over `shot\_layout.usda` and `asset\_lib.usda`. This is the foundation of USD's non-destructive override system.

#7

COMPOSITION • 23% EXAM WEIGHT



What is the key behavioral difference between a `reference` and a `payload` when a stage is opened?

A

References can only reference USDA files; payloads can reference both USDA and USDC

B

References are always loaded immediately when the stage opens; payloads can be deferred and loaded on demand via the stage's load rules

C

Payloads support layer offsets; references do not

D

References compose at the prim level; payloads compose at the layer level

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The fundamental distinction: **References are always loaded** when a stage opens. **Payloads are deferred**—they are only loaded if their prim is included in the stage's load set (controlled via `UsdStage::Open()` load rules or `UsdStage::Load()`/`Unload()`). This is what makes payloads essential for managing working sets in large scenes.

#8

COMPOSITION • 23% EXAM WEIGHT



A pipeline architect wants to let a rigging layer override geometry positions that were established in a modeling layer, while keeping the modeling layer's material assignments intact. Which layer strategy achieves this cleanly?

A

Put the rig layer as a sublayer above the modeling layer; the rig's local opinions for geometry positions will override the modeling layer while material assignments from modeling remain unchanged

B

Use a variant set in the modeling layer with 'rigged' and 'unrigged' variants

C

Flatten both layers into one before rigging begins

D

Reference the modeling layer from inside the rig layer using a payload

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: By placing the **rig layer above the modeling layer** in the sublayer stack, the rig layer's opinions for specific attributes (like point positions and transforms) override the modeling layer. Attributes NOT authored in the rig layer (like material bindings) continue to come through from the modeling layer unchanged. This is the core power of non-destructive layering.

#9

COMPOSITION • 23% EXAM WEIGHT

A USD asset's internal composition includes three nested references. An external stage references this asset. Can the external stage override opinions inside the asset's internal nested references?

A

Yes — the external stage's local opinions are always the strongest and override any internal composition

B

Yes — but only opinions on the asset's defaultPrim can be overridden externally

C

No — the internal composition of a referenced asset is encapsulated; the external stage can override composed values on prims but cannot alter internal composition arc ordering

D

No — nested references are always read-only and cannot be overridden even by local opinions

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Encapsulation means the **internal composition arcs** of a referenced asset cannot be overridden. However, the external stage CAN author **local opinions** on the composed prims (via `over` specifiers), and local opinions are the strongest arc in LIVERPS. So while you cannot change how the asset is composed internally, you CAN override the resulting composed values on any prim.

#10

COMPOSITION • 23% EXAM WEIGHT

An asset has a variant set `material` with variants `plastic` and `metal`. A scene stage selects `metal`. Later, a shot layer also authors a variant selection for `material = plastic` on the same prim. Which selection wins?

A

The scene stage's selection (`metal`) because variant sets are authored in asset files and cannot be overridden

B The shot layer's selection (`plastic`) if the shot layer is stronger (higher) in the sublayer stack than the scene stage layer

C Both variants are composed simultaneously

D USD raises a conflict warning and reverts to the asset's default variant selection

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Variant selections are just metadata opinions and follow standard LIVERPS strength ordering. A **stronger layer's variant selection wins**. If the shot layer is higher in the sublayer stack (or uses a stronger composition arc), its `plastic` selection overrides the scene stage's `metal` selection. This allows shot-level variant overrides.

#11

COMPOSITION · 23% EXAM WEIGHT



Which specifier should be used on a prim that serves as a shared template—one whose opinions should propagate to all prims that inherit from it, but which should NOT appear as a renderable prim itself?

A `def` — standard definition, can be traversed and rendered

B `over` — non-destructive override, used for inheritance templates

C `class` — defines a non-instantiated template prim that is excluded from normal stage traversal

D `payload` — deferred definition prim

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **`class`** specifier creates a prim that acts as a pure template. Class prims are excluded from default stage traversal (they won't be rendered) and exist solely to share opinions with other prims via `inherits`. This is essential for shared look-dev, shared transforms, or any pattern where you want broadcast overrides without adding unwanted geometry to the scene.

#12

COMPOSITION • 23% EXAM WEIGHT

A layer offset of `offset=0, scale=0.5` is applied to a reference. An animation with a time sample at stage time 20 will be evaluated at what local time inside the referenced layer?

- ☒ A Local time 40 — stage time is divided by scale to get local time
- ☐ B Local time 10 — stage time is multiplied by scale to get local time
- ☐ C Local time 20 — scale only affects the offset, not the time value
- ☐ D Local time 0.5 — scale is applied as a fraction of the stage time

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Layer offsets transform from layer-local time to stage time using: $stageTime = localTime \times scale + offset$. Rearranging: $localTime = (stageTime - offset) / scale$. With $offset=0$, $scale=0.5$: $localTime = (20 - 0) / 0.5 = 40$. A scale of 0.5 makes the animation play at half speed on stage (stage needs to advance to time 20 to see what's at local time 10).

#13

COMPOSITION • 23% EXAM WEIGHT

Two assets, `ChairA` and `ChairB`, both reference the same `cushion.usda` asset. A studio wants to update the cushion's UV layout without changing either chair asset. How does this work in USD?

- ☒ A Edit `cushion.usda` directly — since both chairs reference it, both automatically reflect the change when the stage is recomposed
- ☐ B Edit each chair's USD file to include the UV override as a local opinion
- ☐ C The cushion must be duplicated for each chair before any modifications can be made
- ☐ D Changes to `cushion.usda` require a flatten operation on all referencing stages before they take effect

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: This is the core power of USD references. Both `ChairA` and `ChairB` reference the same `cushion.usda`. Editing `cushion.usda` directly causes all stages that reference it to automatically pick up the change upon recomposition. **No changes to the chair files are needed**—this is the 'edit once, update everywhere' benefit of USD's reference-based asset system.

#14

COMPOSITION • 23% EXAM WEIGHT

A prim on a composed stage has opinions from the following sources (listed from strongest to weakest): local layer → variant → reference → specializes. The local layer has NO opinion on attribute `foo`. The variant sets `foo = 1`. The reference sets `foo = 2`. The specializes sets `foo = 3`. What is the composed value of `foo`?

A 1 — from the local layer (even though empty, local always wins)

B 1 — from the variant, the next strongest arc after local

C 2 — from the reference

D 3 — from the specializes, the base class value

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When there is no local opinion, USD moves down LIVERPS to the next strongest arc. **Variants (V)** come after local (L) and before references (E). Since the variant has an opinion (`foo = 1`), it wins over the reference (`foo = 2`) and specializes (`foo = 3`). The composed value is **1**.

#15

COMPOSITION • 23% EXAM WEIGHT

A production pipeline needs multiple artists to collaborate on a complex vehicle asset simultaneously. The vehicle has geometry, rigging, materials, and LODs. What is the most scalable USD layer strategy?

A All artists work in a single shared USD file with strict access controls to prevent conflicts

B Each workstream (geo, rig, materials, LODs) is authored in its own USD layer; these are assembled as sublayers within the asset's main USD file

C Each workstream is stored as a separate variant within the asset's single USD file

D Each workstream creates a complete copy of the asset and the final version is chosen by the lead artist

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The scalable approach is to give each workstream **its own layer** (geo.usda, rig.usda, materials.usda, lod.usda) assembled via sublayers in a main asset file. This enables parallel work without conflicts, supports version control per workstream, and allows the asset's composition to evolve without disrupting other workstreams. This pattern aligns with ASWF asset structure guidelines.

#16

COMPOSITION • 23% EXAM WEIGHT

What happens in USD composition when the same prim path is defined with `def` in two different sublayers of the same stage?

A USD raises a fatal error and refuses to compose the stage

B The second `def` (in the weaker layer) is treated as an implicit `over`—its opinions are composed at the appropriate strength but the prim type from the stronger layer takes precedence

C Both definitions are merged into a single prim with attributes from both layers combined

D The stronger layer's `def` wins entirely; the weaker layer's definition is completely ignored

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD handles multiple `def` specifiers on the same path across sublayers gracefully. The **stronger layer's `def`** establishes the prim type and specifier. The weaker layer's `def` contributes its opinions at the appropriate strength—effectively acting like an `over` in terms of opinion strength. Attributes from both are composed according to LIVERPS.

#17

CONTENT AGGREGATION · 10% EXAM WEIGHT



A crowd simulation outputs 50,000 character positions per frame. The characters all use the same base mesh. Which USD instancing approach is most appropriate for this scale?

A

Native scenegraph instancing with ``instanceable = true`` on each of 50,000 prim paths

B

UsdGeomPointInstancer, which encodes all 50,000 placements as compact attribute arrays on a single prim

C

Reference the character asset 50,000 times using payload arcs for deferred loading

D

Create 50,000 separate USD files, one per character, and reference them all into a master stage

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: UsdGeomPointInstancer is designed exactly for this scale. All 50,000 placements are encoded as arrays (positions, orientations, scales) on a single prim—far more efficient than 50,000 individual scenegraph prims. Native instancing with 50,000 prim paths would still create a huge scenegraph and be expensive to traverse.

#18

CONTENT AGGREGATION · 10% EXAM WEIGHT



In the USD model hierarchy, an ``assembly`` kind prim differs from a ``component`` kind prim in which fundamental way?

A

Assemblies can contain other models (components and assemblies) as children; components are leaf-level models that should not contain other models as children

B

Assemblies use references; components use payloads

C

Assemblies are only used for environments; components are only used for character assets

D

Assemblies are rendered; components are used for logic only

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: **Assembly** is a model kind for aggregating other models—it can contain components, groups, or other assemblies. **Component** is the leaf-level model kind for a self-contained, indivisible asset (a prop, a character). A component should not have other models as children. This hierarchy is enforced by the model kind system and is important for pipeline tools that traverse the model hierarchy.

#19

CONTENT AGGREGATION • 10% EXAM WEIGHT

A scene has a forest of 1,000 trees, all using ``instanceable = true``. A lighting artist needs to make 10 specific trees glow. The most memory-efficient approach that avoids full de-instancing of all 1,000 trees is to:

A

Remove ``instanceable = true`` from all 1,000 trees and add the glow material to each individually

B

Add the glow variant to the prototype and use ``instanceable`` variant-based refinement so only the 10 trees select the glow variant while remaining instanceable

C

Create a new point instancer for just the 10 glowing trees as a separate prim

D

Add a sublayer above the forest layer that sets glow properties on all 1,000 trees

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Using **variant-based refinement** on the prototype is the memory-efficient path. By adding a ``glow`` variant to the prototype, the 10 specific trees can select that variant via their instance prim's metadata—while still sharing the prototype. This avoids de-instancing and keeps the remaining 990 trees fully instanced. This is the 'refinement using variant sets' pattern.

#20

CONTENT AGGREGATION • 10% EXAM WEIGHT

Which attribute on a ``UsdGeomPointInstancer`` maps each instance to one of the defined prototype geometries?

A

``positions`` — the position array implicitly determines which prototype to use based on index

B ``protoIndices`` — an integer array where each element is the index into the ``prototypes`` relationship targets

C ``instanceIds`` — a unique identifier per instance that maps to a prototype

D ``velocities`` — the velocity direction determines which prototype is displayed

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``protoIndices`` is a per-instance integer array where each value is an index into the ``prototypes`` relationship (the list of prototype prim paths). If an instancer has 3 prototype tree variants and ``protoIndices` = [0, 1, 2, 0, 1]`, the first and fourth instances use prototype 0, the second and fifth use prototype 1, and the third uses prototype 2.

#21

CONTENT AGGREGATION • 10% EXAM WEIGHT

A set-dressing artist places 200 lamp posts by referencing the same ``lamp_post.usda`` asset 200 times. All lamps should share geometry memory but the artist has NOT set ``instanceable` = true`. What is the consequence?

A No consequence — USD automatically instances identical references regardless of the ``instanceable`` flag

B USD creates 200 separate copies of the composed prim subtree in memory, losing the memory savings that instancing would provide

C USD refuses to load the stage because too many identical references cause a composition error

D Only the first 10 references are loaded; the rest are deferred as implicit payloads

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Without ``instanceable` = true`, USD creates and caches the composed prim subtree **independently for each reference**—200 copies in memory. Setting ``instanceable` = true` signals to USD that these prims can share a single prototype, reducing memory to roughly one copy plus lightweight per-instance data. Instancing must be explicitly enabled.

#22

CONTENT AGGREGATION · 10% EXAM WEIGHT



An artist needs to add a completely new tree species prototype to an existing `UsdGeomPointInstancer` that already has two prototype trees. What must they update?

A

Only the `positions` array — new positions automatically create new prototype slots

B

Add the new prototype prim as a target to the `prototypes` relationship, then update `protoIndices` for instances that should use the new prototype

C

Create a new separate PointInstancer prim for the new species

D

Update only `protoIndices` — the prototypes relationship auto-discovers new prototype prims

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Adding a new prototype requires two steps: (1) **add the new prototype prim as a target to the `prototypes` relationship** (e.g., append `/Forest/Instancer/Prototypes/TreeOak`), and (2) update **`protoIndices`** for any instances that should display the new prototype (e.g., using index 2 for the newly added third prototype). Both must be updated together.

#23

CONTENT AGGREGATION · 10% EXAM WEIGHT



When using native USD scenegraph instancing, what is an 'instance proxy'?

A

A prim in a separate proxy stage used to preview instanced content without loading the full stage

B

A virtual prim that represents one instance's view into the shared prototype, allowing per-instance overrides to be authored while the prototype remains shared

C

A USD plug-in that manages the synchronization of instance data across multiple machines

D

A metadata attribute that marks a prim as a proxy for a payload arc

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: An **instance proxy** is the virtual representation of a specific instance as seen through the shared prototype. It allows you to query and author per-instance data (like local overrides) while the underlying prim subtree is shared. Authoring significant overrides via an instance proxy may cause de-instancing of that specific instance if the override cannot be represented on the prototype.

CUSTOMIZING USD

#24

CUSTOMIZING USD • 6% EXAM WEIGHT ▲

A studio has created a custom USD schema plugin. Which file must the plugin provide to register itself with USD's plugin registry (`PlugRegistry`) so it is discovered automatically?

A

A `schema.usda` file placed in the USD install directory

B

A `plugInfo.json` file in the plugin's resources directory, declaring the plugin's library path and type registrations

C

A `manifest.xml` file with the plugin's version and compatibility information

D

A `.usdz` archive containing the compiled schema definitions

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's plugin system discovers plugins via `plugInfo.json` files. This JSON file declares: the plugin's shared library path, its type name, the schema types it provides, and any additional metadata. When USD initializes, it scans the `PXR_PLUGINPATH_NAME` environment for directories containing `plugInfo.json` files and registers the found plugins.

#25

CUSTOMIZING USD • 6% EXAM WEIGHT ▲

A team wants to extend USD with a custom file format plugin so that `.sim` files (proprietary simulation format) can be opened directly as USD layers. Which interface must they implement?

A

ArResolver — the asset resolution interface for mapping logical paths to files

B

SdfFileFormat — the interface for implementing custom layer file formats that USD can read and write

C

UsdSchemaBase — the base class for defining new prim types

D

HdRenderDelegate — the Hydra interface for custom renderers

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **SdfFileFormat** is the plugin interface for adding support for new file formats to USD. By implementing ``SdfFileFormat``, a ``.sim`` file can be read, translated to USD scene description on the fly, and treated as a native USD layer. This is how file format plugins like MaterialX, Alembic-to-USD bridges, and others integrate into USD.

#26

CUSTOMIZING USD · 6% EXAM WEIGHT



When should a studio create custom model kinds rather than using the built-in ``component``, ``group``, and ``assembly`` kinds?

A

Always — built-in kinds are deprecated in recent USD versions

B

Only when the studio needs more than 3 levels in the model hierarchy

C

When the studio's pipeline requires domain-specific categorization that the built-in kinds do not capture (e.g., ``fx_element``, ``environment_set``), and these categories need to be queryable by pipeline tools

D

When the asset uses USDC format instead of USDA format

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Custom model kinds are appropriate when a pipeline needs **domain-specific categorizations** beyond the built-in hierarchy. For example, a studio might define ``fx_element`` as a subkind of ``component`` to allow FX-specific pipeline tools to filter and process those assets differently. Custom kinds are defined in a schema plugin and must be a subkind of an existing kind to maintain hierarchy integrity.

#27

CUSTOMIZING USD • 6% EXAM WEIGHT



A Hydra ScenelIndex plugin is different from a traditional USD plugin in what key way?

A

ScenelIndex plugins operate on SdfLayer data before it reaches the UsdStage

B

ScenelIndex plugins intercept and transform scene data in the Hydra rendering pipeline, generating or modifying render-ready prims without changing the USD scene description

C

ScenelIndex plugins can only be written in C++ and not in Python

D

ScenelIndex plugins replace the need for custom USD schemas

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **Hydra ScenelIndex plugin** operates at the rendering layer—it intercepts the stream of 'Hydra prims' that are sent to the renderer and can add, remove, or modify them procedurally. Unlike USD schema plugins that add data at the scene description level, ScenelIndex plugins modify what the renderer sees *without changing the underlying USD scene description*. This enables procedural geometry generation and rendering-specific data injection.

DATA EXCHANGE

#28

DATA EXCHANGE • 15% EXAM WEIGHT



A pipeline receives a USD asset from an external partner. The `usdchecker` tool reports: 'Prim /World has no valid defaultPrim.' What is the impact of this warning for downstream pipeline stages?

A

The prim cannot be rendered—renderers require a valid defaultPrim to function

B

Any pipeline stage that references this asset without specifying an explicit prim path will fail to compose the reference, because USD cannot determine the reference root

C

The warning is cosmetic only—composition always succeeds regardless of defaultPrim

D

The asset will be loaded at double scale until defaultPrim is set

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Missing ``defaultPrim`` means that any downstream stage that references this asset **without an explicit prim path** will experience a composition failure—USD cannot determine what to use as the root of the reference. This is a critical error for referenceable assets and must be fixed before the asset can be safely used in production pipelines.

#29

DATA EXCHANGE · 15% EXAM WEIGHT

A DCC exporter writes a USD file where all transforms are stored as 4×4 matrices (``xformOp:transform``) rather than decomposed scale/rotate/translate operations. A downstream tool only supports ``xformOp:translate``, ``xformOp:rotateXYZ``, and ``xformOp:scale``. What problem does this create?

A

No problem — USD automatically decomposes matrix transforms into separate operations

B

The downstream tool cannot read the ``xformOp:transform`` attribute and will display the asset at the origin with default transform values

C

The matrix format causes a USDC parsing error in the downstream tool

D

The downstream tool will interpret the matrix as a primvar

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: USD stores all transforms as authored xformOps. A tool that only supports specific xformOp types will be unable to interpret ``xformOp:transform`` (the 4×4 matrix form). It will likely fall back to identity transform, placing the asset at the origin. This is a **data exchange fidelity issue**—exporters should author transforms in the most broadly compatible decomposed form unless the receiver is known to support matrices.

#30

DATA EXCHANGE · 15% EXAM WEIGHT

A pipeline is implementing a USD exporter from a proprietary DCC. The DCC has a 'multi-material' mesh where different polygon groups have different materials. How should this be represented in USD?

A Create a separate Mesh prim for each material region and bind materials to each Mesh

B Use UsdGeomSubset to partition the mesh's faces into named subsets, then bind different materials to each subset using the MaterialBindingAPI

C Store the material assignments as a primvar array with one material path per face

D Use variant sets to swap between different material assignments on the whole mesh

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **UsdGeomSubset** is the correct USD mechanism for multi-material meshes. Each subset defines a partition of the mesh faces (as a face index array), and 'UsdShadeMaterialBindingAPI' can bind a different material to each subset. This is the USD equivalent of material groups in other 3D formats and is the approach used by USD's own compliance guidelines.

#31

DATA EXCHANGE • 15% EXAM WEIGHT



When performing a conceptual data mapping between USD and another 3D format, what is the purpose of the mapping document?

A To specify which programming language the importer/exporter must be written in

B To formally define how concepts in the source format correspond to USD constructs—identifying equivalences, lossy conversions, and untranslatable features to guide importer/exporter design

C To provide a list of all USD API functions needed by the converter

D To document the file size difference between the source format and USD

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **conceptual data mapping document** formally describes how each feature of the source format maps to USD (or vice versa). For example: 'Maya's shading groups → UsdGeomSubset + MaterialBindingAPI; Maya's instancer → UsdGeomPointInstancer.' It identifies: direct equivalences, partial mappings with data loss, and features with no USD

equivalent. This document guides both importer and exporter implementation and sets stakeholder expectations about fidelity.

#32

DATA EXCHANGE · 15% EXAM WEIGHT

A USD file uses `asset` type attributes to reference textures. After delivering the USD file to a vendor, they report they cannot find the textures. The paths in the file are `./textures/diffuse.png`. What is the most likely issue?

A

The `asset` attribute type is not supported in USDC format, only USDA

B

The vendor received only the USD file but not the texture files — relative paths require the referenced files to be present alongside the USD file or in a USDZ package

C

Relative paths are not valid in USD asset attributes

D

The textures must be embedded directly in the USD file using base64 encoding

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Relative asset paths like `./textures/diffuse.png` are resolved relative to the USD layer file that contains them. If the vendor only received the `.usda` file without the `textures/` directory, the paths cannot resolve. **External delivery should use USDZ** (which bundles all assets) or explicitly include all referenced files. This is a common data exchange oversight.

#33

DATA EXCHANGE · 15% EXAM WEIGHT

The `usdcat` command-line tool is primarily used for which purpose?

A

Validating USD files for schema compliance

B

Rendering a USD stage to an image

C

Converting between USD file formats (e.g., USDA to USDC) and printing composed stage content

D

Packaging USD files and their dependencies into a USDZ archive

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: `usdcat` reads USD layers and can: print the content as USDA text (even binary USDC files), flatten a composed stage to a single layer (`--flatten`), and convert between formats by writing output in a different format. It's the essential command-line tool for inspecting USD content and performing format conversions.

#34

DATA EXCHANGE · 15% EXAM WEIGHT

A glTF-to-USD converter must handle glTF's ``KHR_materials_pbrMetallicRoughness`` extension. What is the most appropriate USD material to map this to?

A

A raw `UsdShadeShader` prim with custom attributes for each PBR parameter

B

`UsdPreviewSurface`, which provides metallic, roughness, baseColor, and other PBR parameters that align closely with glTF's metallic-roughness model

C

`UsdGeomMesh` material binding using a ``primvars:displayColor`` primvar

D

A MaterialX standard surface node network

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: `UsdPreviewSurface` is USD's built-in PBR shading model specifically designed for interoperability. Its ``metallic``, ``roughness``, ``diffuseColor``, ``opacity``, and other inputs align closely with glTF's metallic-roughness workflow. It is the recommended mapping target for PBR materials during glTF-to-USD conversion for broad tool compatibility.

#35

DATA EXCHANGE · 15% EXAM WEIGHT

A studio is writing a USD importer for a DCC. The source data contains instanced geometry where multiple objects share the same mesh data. How should this be represented in USD to preserve the instancing relationship?

A

Inline the mesh data into each object's USD prim separately, since USD doesn't support instancing from importers

B Reference a shared mesh USD file from each instance prim and set ``instanceable = true`` on each, allowing USD to share the prototype

C Store all instances as children of a single Xform prim with a shared material binding

D Use ``UsdGeomMesh`` with the same ``points`` array for each instance

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: To preserve instancing, each instance prim should **reference the shared mesh asset** (or a shared layer), and all should have ``instanceable = true``. USD will recognize they reference the same content and create a shared prototype. This mirrors the original instancing relationship in the source data and is memory-efficient.

#36

DATA EXCHANGE • 15% EXAM WEIGHT



When exporting animation from a DCC to USD, what is the recommended approach for encoding time-sampled vertex positions on a deforming mesh?

A Store vertex positions at every frame as separate attributes with frame numbers in their names

B Use USD time samples on the ``points`` attribute—author a time sample for each frame (or sub-frame) of the animation

C Encode the entire animation as a string attribute containing a JSON array of positions per frame

D Use variant sets with one variant per frame of animation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's native mechanism for animated data is **time samples** on attributes. For deforming meshes, the ``points`` attribute should have a time sample for each animated frame. USD's interpolation system (held or linear) handles in-between frame queries. This is the standard approach for DCC-to-USD mesh animation export.

#37

DATA EXCHANGE · 15% EXAM WEIGHT



A pipeline needs to validate that all exported USD assets have correct `extent` attributes. Which USD API method is designed to compute the correct extent for a boundable prim?

A

`UsdGeomBoundable::ComputeExtentFromPlugins()` — computes extent using registered extent computation plugins for the prim's type

B

`UsdGeomMesh::ComputeBoundingBox()` — returns the world-space bounding box

C

`UsdStage::ComputeWorldBound()` — computes the world-space bound of the entire stage

D

`UsdPrim::ComputeLocalBound()` — computes the local-space bound without needing plugins

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdGeomBoundable::ComputeExtentFromPlugins()`` dispatches to type-specific extent computation (registered via plugins) to compute the correct bounding box for a given prim type. This ensures the extent is computed according to the prim's schema—e.g., a Mesh computes from its points, a Sphere from its radius. Validators should use this to verify or recompute extents after DCC export.

#38

DATA EXCHANGE · 15% EXAM WEIGHT



A USD file is delivered with `metersPerUnit = 1.0` (meters). A downstream renderer expects all scenes in centimeters. What is the correct pipeline handling for this mismatch?

A

Reload the file in a text editor and multiply all numeric values by 100

B

Apply a uniform scale of 100 on the root Xform prim in a shot layer, and update `metersPerUnit` to 0.01 in the root layer to reflect the new convention

C

USD automatically applies the scale factor at render time—no action needed

D

Convert the file to USDC format, which stores units in centimeters by default

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: The correct approach is to apply a **compensating scale transform** (scale = 100 for meters-to-centimeters) on a root Xform prim in a stronger layer, and optionally update ``metersPerUnit`` in the stage's root layer to ``0.01``. This preserves the original asset data, maintains traceability, and correctly represents the scale relationship in the composed stage.

#39

DATA MODELING • 15% EXAM WEIGHT

A ``UsdGeomMesh`` has ``points`` that are animated over 100 frames. The ``normals`` attribute is NOT animated. What is the expected rendering behavior?

A

The normals are recomputed from the animated points automatically by USD at each frame

B

The static normals are used at every frame regardless of point animation—potentially causing incorrect shading on deformed frames

C

USD raises an error if normals are not animated when points are animated

D

The renderer uses the default smooth normals when authored normals are not animated

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: USD does NOT automatically recompute normals from animated points. **Static normals remain fixed** even as geometry deforms, causing incorrect surface shading. For deforming meshes, normals must either be animated in sync with points, or left unset so the renderer computes them from the deformed geometry. This is a common DCC export oversight.

#40

DATA MODELING • 15% EXAM WEIGHT

What is the purpose of ``xformOpOrder`` on a ``UsdGeomXformable`` prim?

A

It specifies the render order of the prim relative to its siblings

B

It defines the ordered list of `xformOp` attributes to apply when computing the prim's local transform, and the order matters because matrix multiplication is not commutative

C

It sets the priority of transform operations relative to parent prim transforms

D

It controls which xformOp values are time-sampled versus static

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `xformOpOrder` is a token array that specifies both *which* xformOp attributes contribute to the local transform AND *the order* in which they are applied. Order matters critically because matrix multiplication is not commutative—`translate then rotate` produces a different result than `rotate then translate`. This explicit ordering prevents ambiguity when multiple transform ops are present.

#41

DATA MODELING • 15% EXAM WEIGHT



A developer needs to store per-face material IDs as integers on a mesh. Which is the most appropriate USD data modeling choice?

A

A `uniform int[]` primvar with one value per face

B

A `float[]` attribute named `facelds` in the `userProperties:` namespace

C

A relationship from the mesh to a list of material prims

D

A `token[]` attribute in the mesh's metadata

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Per-face integer data is best stored as a `uniform int[]` primvar. `uniform` interpolation means one value per face, which matches per-face material IDs exactly. Using a primvar (rather than a plain attribute) provides the interpolation metadata that renderers and pipeline tools need to correctly interpret the data. The `int` type is appropriate for integer IDs.

#42

DATA MODELING • 15% EXAM WEIGHT



A prim has both a `default` value and `timeSamples` authored for the same attribute. An application queries the attribute at timeCode 0 (assuming the first time sample is at time 10). What value does USD return?

A The default value — because timeCode 0 is before the first time sample

B An interpolated value between the default and the first time sample

C 0.0 — USD returns zero when queried before the first time sample

D The value of the first time sample (at time 10) — USD snaps to the nearest sample

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When queried before the first time sample (and with default ``UsdInterpolationType::Linear`` or ``Held``), USD returns the **default value** for time codes before the first sample. Default values act as the 'held' value for the region before any time samples begin. The default and time samples are separate data sources; USD does NOT interpolate between them.

#43

DATA MODELING • 15% EXAM WEIGHT

A pipeline engineer is deciding whether to store a shader's node graph connections as USD relationships or as USD attributes. Which is correct?

A Connections in UsdShade node graphs are stored as attributes of type ``token`` referencing shader node names

B UsdShade uses a dedicated connection API (``UsdShadeConnectableAPI``) to create connections, which are internally represented as attribute connections (source/target paths) rather than relationships

C Shader connections are stored as USD relationships pointing to the connected shader prim's path

D Shader connections are not representable in USD and must be stored in sidecar JSON files

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **UsdShade** uses **attribute connections** (not relationships) to represent shader network edges. ``UsdShadeConnectableAPI::ConnectToSource()`` creates connections between shader inputs and outputs, stored as connection metadata on attributes. This allows the full

shader graph to be represented natively in USD with type checking and proper composition behavior.

#44

DATA MODELING • 15% EXAM WEIGHT

A mesh has 4 vertices and 1 quad face. A primvar with `vertex` interpolation has 4 values. A separate primvar with `faceVarying` interpolation needs to store different UV values at each corner of the quad. How many values should the `faceVarying` primvar have?

- A 1 — one value for the single face
- B 4 — one value per unique vertex
- C 4 — one per face-vertex (4 corners of 1 quad), which equals the vertex count in this case**
- D 8 — two UV coordinates per corner

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: For this mesh: 1 quad × 4 corners = **4 face-vertex values** for `faceVarying` interpolation. In this specific case the count happens to equal the vertex count, but for a multi-face mesh they differ. `faceVarying` always stores one value per face-vertex (each corner of each face), regardless of whether vertices are shared. For the UV attribute `texCoord2f`, each value is a `(u,v)` pair.

#45

DATA MODELING • 15% EXAM WEIGHT

What is the `token` value type used for in USD scene description?

- A A numeric token ID used for fast attribute lookup
- B An interned (deduplicated) string used for enumerated values, identifiers, and metadata values — more efficient than raw strings for repeated values**
- C A binary blob that stores arbitrary data types

D A reference to a prim path in the scene graph

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``token`` in USD is an interned string—meaning equal strings are stored only once in memory and compared by pointer identity rather than character-by-character. Tokens are used for enumerated values (like ``visibility = 'invisible'``), schema type names, namespace prefixes, and other repeated string values. They are more efficient than ``string`` for frequently repeated identifiers.

#46

DATA MODELING • 15% EXAM WEIGHT



A mesh has both ``normals`` and ``primvars:normals`` defined. Which one does a Hydra-based renderer use for shading?

A ``normals`` — the non-primvar attribute always takes precedence

B ``primvars:normals`` — renderers prefer primvar normals because they carry interpolation metadata

C The renderer uses an average of both

D USD raises an ambiguity error and falls back to computed smooth normals

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When both are present, **Hydra-based renderers prefer ``primvars:normals``** because it carries explicit interpolation metadata (vertex, faceVarying, etc.) that is essential for correct shading. The raw ``normals`` attribute (inherited from older USD schemas) lacks this metadata. Modern USD best practice is to always use ``primvars:normals`` for authored normals.

#47

DATA MODELING • 15% EXAM WEIGHT



A pipeline stores asset version information on prim metadata. This version must survive flattening and be readable by any tool without schema plugins. Which metadata storage approach is most appropriate?

A Store it as a typed attribute in the `userProperties:` namespace

B Store it as a `assetInfo` metadata dictionary on the prim — a built-in, untyped metadata dictionary specifically for asset management information

C Store it as a variant selection in a `version` variant set

D Store it in the layer's `customLayerData` as a nested dictionary

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `assetInfo` is a built-in USD metadata dictionary on prims specifically intended for asset management information like version, identifier, and name. Unlike custom metadata that requires schema registration, `assetInfo` is a standard USD feature available without plugins, survives flattening, and is the canonical location for pipeline-readable asset metadata.

DEBUGGING

#48

DEBUGGING • 11% EXAM WEIGHT

An artist changes a variant selection in `usdview` but the scene doesn't update. The stage appears frozen. What is the most likely cause and solution?

A Variant selections require a full application restart to take effect in usdview

B The stage may be using an older composition cache — refreshing/reloading the stage in usdview will trigger recomposition with the new variant selection

C Variant selections cannot be changed interactively; they must be set before the stage opens

D The prim's `active` metadata is false, preventing variant changes from taking effect

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In interactive tools like `usdview`, changing a variant selection should trigger automatic recomposition. If the stage appears frozen, the most likely issue is a **stale composition cache** or a display refresh issue. Reloading the stage (or refreshing the view)

forces USD to recompose with the new variant selection. In rare cases, a bug in the DCC or viewer may require restarting.

#49

DEBUGGING · 11% EXAM WEIGHT

A composed mesh has the correct shape but appears with a completely black material in the renderer. Materials are defined in the stage. What should you check first?

A Verify the mesh's `extent` attribute is correctly computed

B Check that the `MaterialBindingAPI` schema is applied to the mesh prim and that the `material:binding` relationship points to a valid material prim path

C Confirm the mesh's `normals` attribute is authored

D Check that the mesh's `purpose` is set to `render`

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A black material in the renderer most commonly indicates a **missing or broken material binding**. The mesh prim needs `MaterialBindingAPI` applied (as an apiSchema), and the `material:binding` relationship must point to a valid, existing material prim. If the path is broken (e.g., the material prim was moved or renamed), the renderer has no material to use and falls back to a default shader.

#50

DEBUGGING · 11% EXAM WEIGHT

After muting a layer in a stage, a developer notices that some attribute values changed unexpectedly on prims that are NOT defined in the muted layer. Why might this happen?

A Muting a layer clears the composition cache for the entire stage, causing all values to revert to defaults

B The muted layer may have been contributing variant selections or other composition metadata that affected which opinions were active across the stage

C Muting a layer propagates to all sublayers of the muted layer, removing additional opinions

D USD randomly invalidates attribute values when layers are muted to prevent data corruption

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Muting a layer removes ALL of its scene description from composition, including **variant selections, sublayer references, and other composition metadata**. If the muted layer was selecting a variant on another prim, muting it changes the active variant, which can change attribute values across the stage—even on prims that aren't directly defined in the muted layer.

#51

DEBUGGING • 11% EXAM WEIGHT

A USD stage opens without errors but takes 45 seconds. Investigation shows the bottleneck is in asset resolution. Which approach is most likely to improve performance?

A Convert all USDA layers to USDC format

B Investigate the custom ArResolver — it may be making synchronous network calls or database queries for every asset path; add caching to the resolver

C Reduce the number of prims in the stage by merging meshes

D Flatten the stage before opening it to eliminate composition overhead

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: If the bottleneck is in asset resolution, the custom **ArResolver** is the primary suspect. Common issues include: synchronous network/database calls for every path (should be batched or cached), missing resolution cache invalidation, or redundant resolution of the same paths. Adding an in-memory **cache** to the resolver is typically the highest-impact fix for resolution performance.

#52

DEBUGGING • 11% EXAM WEIGHT

A prim at `/World/Car/Door` exists in the composed stage but is not returned by a stage traversal. What is the most likely cause?

- ☐ A The prim's `kind` metadata is set to `subcomponent`
- ☐ B The prim or one of its ancestors has `active = false`, which prunes it from traversal
- ☐ C The prim is defined with the `class` specifier and class prims are skipped by default traversal
- ☒ D Both B and C are valid causes

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Two common reasons a prim is skipped during traversal: (1) `active = false` — deactivating a prim prunes it and all descendants from default stage traversal; (2) `class specifier` — class prims are excluded from normal traversal predicates. Both can cause a prim to exist in the stage but be invisible to traversal. Always check both when debugging missing prims.

#53

DEBUGGING · 11% EXAM WEIGHT

A geometry cache tool reports that object positions are drifting over time for a specific character, even though no animation is authored on the position. What USD-related issue could cause this?

- ☐ A The character prim has `instanceable = true`, causing its position to be shared and overwritten
- ☒ B A value clip on the stage is loading time-sampled position data from an unexpected clip layer, contributing motion to the position attribute
- ☐ C The `faceVertexIndices` attribute is being incorrectly reinterpreted as position data
- ☐ D The character's `extent` attribute is animating, which forces position recalculation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Value clips load time-sampled data from external clip layers, and if clip metadata (clipAssetPaths, clipPrimPath, clipActive) is accidentally applied to a prim, it can inject unexpected time-sampled data—including position motion—even if no animation was intentionally authored. Checking the prim's clip metadata is an important debugging step for unexplained animation.

#54

DEBUGGING • 11% EXAM WEIGHT



When would you use ``UsdStage::MuteLayer()`` during debugging rather than simply removing the layer from the sublayer list?

A

When the layer needs to be permanently removed from the pipeline

B

``MuteLayer()`` is non-destructive — it temporarily excludes a layer from composition without modifying any USD files, allowing quick isolation of which layer is causing a problem

C

When the layer contains binary USDC data that cannot be deleted

D

``MuteLayer()`` is the only way to remove a layer's influence when it is added via a reference rather than a sublayer

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdStage::MuteLayer()`` is a non-destructive debugging tool. It excludes a layer from composition temporarily—at runtime, without modifying any files. This makes it ideal for quickly testing 'what if this layer didn't exist?' scenarios, isolating problematic layers, or understanding which layer is contributing an unexpected opinion. The stage recomposes without the muted layer.

#55

DEBUGGING • 11% EXAM WEIGHT



A pipeline tool that subscribes to ``UsdNotice::ObjectsChanged`` is receiving thousands of notifications per second, causing performance degradation. What is the best architectural solution?

A

Unsubscribe from all notices and poll the stage for changes every 5 seconds

B Wrap batch edits in ``SdfChangeBlock`` to consolidate change notifications; debounce the callback to coalesce rapid-fire notifications before processing

C Switch to a weaker notice type that only fires once per stage open

D Move all stage editing to a background thread to prevent notification delivery on the main thread

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Two complementary solutions: (1) ``SdfChangeBlock`` coalesces changes during batch edits so only one notice is sent per block; (2) **debouncing** the callback (only processing it after a brief idle period) prevents cascading updates from a torrent of individual changes. Together these prevent notification storms while preserving correctness.

PIPELINE DEVELOPMENT

#56

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline has assets from multiple DCCs with inconsistent naming conventions for prim hierarchies (e.g., Maya exports ``/pSphere1/pSphereShape1``; Houdini exports ``/geo/mesh``). What is the recommended long-term solution?

A Accept all naming conventions and build converters to handle each case

B Define and enforce a studio-wide USD prim naming convention via exporter hooks or post-export transforms, normalizing all assets to a consistent hierarchy before they enter the asset library

C Use variant sets to toggle between each DCC's native naming convention

D Store DCC-specific naming as metadata and let downstream tools interpret it

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Inconsistent naming creates fragile assumptions in downstream pipeline tools. The best solution is to define **studio-wide naming conventions** and enforce them via **exporter hooks**

or **post-export USD transforms** that normalize the prim hierarchy before assets enter the library. This ensures all tools can make consistent assumptions about asset structure.

#57

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A USD pipeline needs to handle the scenario where an artist accidentally publishes a new version of an asset with a broken reference. What pipeline practice prevents this from reaching downstream stages?

A Use payload arcs instead of references so the broken asset only fails when loaded

B Implement a publish-time asset validation step that runs `usdchecker` and resolves all references before accepting a publish

C Require artists to manually inspect all referenced files before publishing

D Use USDC format for all published assets to prevent reference path errors

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **publish-time validation step** using `usdchecker` (and custom validators) that resolves all references before accepting a publish is the industry best practice. This creates a quality gate that catches broken references, missing assets, missing defaultPrim, incorrect extents, and other issues before they propagate to downstream stages.

#58

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A studio is designing a shot pipeline where multiple departments (layout, animation, lighting, FX) need to contribute opinions to the same shot stage. What is the recommended architecture?

A Each department writes to a separate shot file; the lead TD manually merges them before rendering

B All departments share a single shot USD file and use a version control lock system to prevent simultaneous edits

C A root shot layer sublayers department-specific layers in a defined strength order; each department edits only their layer

D Each department creates a separate stage and all stages are rendered independently then composited in post

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The industry-standard shot pipeline uses a **root shot layer** that sublayers department-specific layers in a defined strength order (e.g., FX > lighting > animation > layout). Each department edits only their own layer. This enables parallel work, clear ownership, correct opinion priority, and non-destructive override workflows without conflicts.

#59

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline engineer is deciding whether to use ``UsdStage::Export()`` or ``UsdStage::Flatten()`` for delivering a finalized asset. What is the key difference?

A ``Export()`` saves only the root layer; ``Flatten()`` saves all sublayers as separate files

B ``Flatten()`` evaluates all composition and writes a single flat layer with no remaining composition arcs; ``Export()`` saves the composed stage to a file but preserves some structural metadata

C Both are functionally identical — use whichever is more convenient

D ``Export()`` requires USDC format; ``Flatten()`` requires USDA format

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``Flatten()`` evaluates all composition (resolves all variants, references, payloads, inherits, sublayers) and produces a **single layer with no composition arcs**—the fully-baked result. ``Export()`` writes the stage to a file while potentially preserving layer structure. For external delivery where recipients lack access to pipeline resolvers, ``Flatten()`` is the correct choice.

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

#60

A pipeline is adopting USD and needs to version assets. An asset ``chair_v001.usda`` is referenced by a shot. When ``chair_v002.usda`` is published, how should version switching be managed without editing the shot file?

- A Rename ``chair_v002.usda`` to ``chair_v001.usda`` to replace the old version in-place
- B Use a logical asset path (e.g., ``asset://props/chair.usda``) resolved by a custom `ArResolver` that maps logical names to versioned files; version switching happens in the resolver's configuration**
- C Add a new reference to ``chair_v002.usda`` in the shot file as a prepended reference
- D Store both versions as variants in the shot layer and let artists choose

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Using **logical asset paths** with a custom **ArResolver** is the professional pipeline approach. The shot file references ``asset://props/chair.usda``, and the resolver's version configuration determines which physical file that maps to. Switching versions requires only updating the resolver's version pinning—**no shot files need to be edited**. This is how asset management systems like ShotGrid/Flow integrate with USD.

#61

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline generates USD stages programmatically and the tool needs to know when composition is complete before reading any attribute values. Which approach ensures composition is evaluated before querying?

- A Call ``UsdStage::Reload()`` before any attribute queries to force recomposition
- B USD composition is always evaluated lazily and synchronously when you call attribute query APIs — no explicit 'wait for composition' call is needed**
- C Subscribe to ``UsdNotice::StageContentsChanged`` and wait for the notification before querying
- D Call ``UsdStage::GetPseudoRoot().ComposePrimChildren()`` to pre-compose all prims

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: USD evaluates composition **lazily and synchronously**. When you call ``UsdAttribute::Get()``, ``UsdPrim::GetAttribute()``, or any other query API, USD evaluates the necessary composition on demand and returns the result synchronously. There is no need for explicit 'wait for composition' calls in single-threaded contexts—the API handles composition evaluation automatically.

#62

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A studio documents their asset structure guidelines and specifies that all component assets must have a ``extentsHint`` authored on the root prim. Why is ``extentsHint`` important for pipeline tools?

A

``extentsHint`` is required for the asset to be renderable in any USD renderer

B

``extentsHint`` provides a pre-computed bounding box per purpose (render, proxy, guide) that tools can read without loading the asset's payload geometry, enabling fast scene layout and LOD selection

C

``extentsHint`` is used by the `ArResolver` to determine the asset's physical file size

D

``extentsHint`` specifies the asset's coordinate system and unit scale

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: ``extentsHint`` (from ``UsdGeomModelAPI``) stores pre-computed bounding box extents per imageable purpose (render, proxy, guide) on the model's root prim. Pipeline tools and set-dressing applications can read ``extentsHint`` *without loading the asset's payload*, enabling fast bounding box queries, scene layout, camera framing, and LOD decisions at the asset level—critical for large-scale scenes.

#63

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline uses USD's ``UsdPhysics`` schemas to store physics properties on assets. Why should these physics attributes be stored in a separate sublayer rather than in the geometry layer?

A USD requires physics data in a separate layer for licensing reasons

B Keeping physics in a separate sublayer allows physics simulation to be added or removed without modifying the base geometry asset; it enables workflow separation and cleaner pipeline responsibilities

C Physics attributes are incompatible with geometry attributes in the same layer

D Physics layers must always use USDA format while geometry layers use USDC

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Separating physics data into its own sublayer is a **workflow separation** best practice. The geometry team owns the geo layer; the physics team owns the physics layer. This means: physics can be added/removed without touching the geo asset, different pipeline stages can include/exclude physics data, and the ownership and responsibilities are clearly defined. This mirrors how lighting, animation, and FX are separated in shot pipelines.

#64

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline tool needs to find all assets in a stage that are published components (kind = component) without loading any payloads. What API approach achieves this?

A Traverse the stage with payloads loaded and filter by kind metadata

B Use ``UsdPrimRange`` with a predicate that includes unloaded prims, then check ``prim.GetKind()`` (via ``UsdModelAPI``) for each prim in the traversal

C Query the stage's root layer for all prims that have ``kind = component`` metadata

D Use ``UsdStage::GetMaster()`` to get all component prototypes

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Using ``UsdPrimRange`` with predicates that visit unloaded prims (e.g., ``UsdPrimDefaultPredicate`` adjusted to include inactive or unloaded), then checking ``UsdModelAPI(prim).GetKind()`` for each prim, allows finding components without loading

payloads. The `kind` metadata is authored on the prim (outside any payload arc) specifically so it can be queried without payload loading.

#65

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

When should a pipeline use ``UsdStage::SaveSessionLayer()`` versus ``UsdStage::Save()``?

A

``SaveSessionLayer()`` saves the root layer; ``Save()`` saves all sublayers

B

``Save()`` saves all dirty layers to their respective files; ``SaveSessionLayer()`` persists the session layer, which holds temporary interactive overrides (like variant selections made in a DCC)

C

Both methods are identical — they save all modified layer data to disk

D

``SaveSessionLayer()`` is for USDA format; ``Save()`` is for USDC format

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **session layer** is a special, anonymous layer at the top of the layer stack for temporary interactive overrides (viewport-level changes, debug overrides, interactive variant selections). ``SaveSessionLayer()`` explicitly persists this temporary layer. ``Save()`` saves all dirty layers (root and sublayers) to their respective files but does NOT save the session layer by default.

VISUALIZATION

#66

VISUALIZATION • 8% EXAM WEIGHT

A render engineer notices that an asset looks correct in a Hydra Storm viewport but incorrect in a path-traced renderer (Karma, Arnold, RenderMan). The asset uses ``UsdPreviewSurface`` only. What is the most likely cause?

A

Path tracers do not support USD assets — they require conversion to proprietary formats

B UsdPreviewSurface is designed for real-time preview; production renderers use their own native shading networks. The asset likely lacks renderer-specific material networks (e.g., MaterialX or renderer-specific shaders) that the production renderer expects

C The asset's USDC format is incompatible with production renderers

D The `purpose` attribute is set to `proxy` instead of `render`

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **UsdPreviewSurface** is a *preview* shader designed for real-time Hydra Storm rendering. Production renderers (Arnold, RenderMan, Karma) use their own native shading networks. A production pipeline should provide **renderer-specific material networks** alongside or instead of UsdPreviewSurface, typically using MaterialX or renderer-specific schema. Assets with only UsdPreviewSurface may render differently or fall back to a default shader in production renderers.

#67

VISUALIZATION • 8% EXAM WEIGHT

The `purpose` attribute on a `UsdGeomImageable` prim can have which values, and what is the effect of each?

A `visible` and `invisible` — controlling whether the prim is drawn

B `render`, `proxy`, `guide`, and `default` — controlling which geometry is sent to which rendering contexts (final render vs. viewport proxy vs. guide overlays)

C `high`, `medium`, and `low` — defining LOD levels for the geometry

D `static` and `dynamic` — indicating whether the geometry moves in the scene

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **`purpose`** on **`UsdGeomImageable`** has four values: **`default`** (always drawn), **`render`** (production render only), **`proxy`** (lightweight viewport stand-in), and **`guide`** (non-rendered helper geometry for rigging/animation). This allows assets to carry multiple

representations—a complex render mesh, a simple proxy sphere, and guide curves—with renderers and viewports automatically selecting the appropriate geometry.

#68

VISUALIZATION · 8% EXAM WEIGHT

A UsdLux `RectLight` is placed in a scene but it illuminates in the wrong direction. What is the correct way to re-orient it?

A

Change the `texture:format` attribute to flip the light direction

B

Set `inputs:spread` to a negative value to reverse the emission direction

C

Apply a rotation transform to the `RectLight` prim — UsdLux lights inherit from `UsdGeomXformable` and are oriented via their transform

D

Use a `LightFilter` prim to redirect the light direction

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: All UsdLux light types inherit from `UsdGeomXformable`, meaning they are oriented using standard transform operations (rotations, translations). A `RectLight` emits in the local -Z direction by default. To re-orient it, apply a rotation xformOp to the light prim. This is consistent with how all scene objects are positioned and oriented in USD.

#69

VISUALIZATION · 8% EXAM WEIGHT

A mesh renders correctly in isolation but appears visually wrong when placed inside a grouped Xform hierarchy. The mesh positions appear offset. What is the most likely cause?

A

Meshes cannot be placed inside Xform prims in USD

B

The parent Xform prim has an unintended local transform (e.g., residual translate or scale from authoring), and USD computes the mesh's world transform by concatenating all ancestor transforms

C

The mesh's `extent` attribute is in world space and conflicts with the local-space geometry

D

The mesh's material binding is being inherited from the parent Xform

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: USD computes a prim's world transform by **concatenating all ancestor transforms**. If a parent Xform has an unintended local transform (left over from authoring, DCC import, or an incorrectly authored xformOp), all child geometry will appear offset in world space. The fix is to inspect and zero out the parent Xform's transform operations, or use ``UsdGeomXformCache`` to diagnose the full transform stack.

#70

VISUALIZATION • 8% EXAM WEIGHT



A material is bound to a Mesh prim with ``bindMaterialAs = 'weakerThanDescendants'``. A child UsdGeomSubset of the mesh has its own material binding. Which material is used for the subset's faces?

A

The material bound to the parent Mesh, because parent bindings always take precedence

B

The material bound to the UsdGeomSubset, because subset bindings are always stronger

C

The Subset's material wins because the parent Mesh's binding was explicitly set to ``weakerThanDescendants``, making it weaker than the Subset's binding

D

Both materials are blended together using the subset's face weighting

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: ``bindMaterialAs = 'weakerThanDescendants'`` is an explicit instruction that the parent prim's binding should yield to any binding on descendant prims (including UsdGeomSubsets). This allows a mesh-level material to serve as a fallback while subsets override specific face regions with their own materials—the intended multi-material binding pattern.